

Employee Custody Management — User Guide

odomate_hr_custody · Odoo 19.0 · version 19.0.1.0.0

Track every piece of company property you hand to an employee: a register of items, a request/approval/return workflow, automatic overdue chasing, and a printable handover document — all inside the **Employees** app.

Table of contents

1. What this module does
2. Installation
3. Roles and permissions
4. Where to find it
5. Managing the item register
6. The request workflow
7. Extending a return date
8. Refusals
9. Overdue reminders
10. The handover document
11. Analysis and employee smart buttons
12. Field reference
13. Limitations

1. What this module does

The module adds two persistent models and two wizards:

Model	Purpose
<code>odomate.hr.custody.item</code>	The register entry: one row per physical item you lend out.
<code>odomate.hr.custody</code>	The request: who has the item, why, and until when.
<code>odomate.hr.custody.extend</code>	Wizard used by the holder to propose a later return date.
<code>odomate.hr.custody.refuse</code>	Wizard used by HR to refuse a request or an extension, with a reason.

It also adds two smart buttons to the **hr.employee** form.

2. Installation

`odomate_hr_custody` depends on `hr`, `mail`, `product` and `odomate_hr_employee_info_06092026_2`.
 Odoo installs any of those that are missing automatically.

1. **Apps** → **Update Apps List**.
2. Search for Employee Custody Management.
3. Click **Install**.

No post-install configuration is required. The reference sequence, the reminder email template and the daily cron are created by the install.

3. Roles and permissions

The module reuses the standard Odoo HR groups — it does not create groups of its own.

Group	Items	Requests	Approve / Refuse / Returned	Analysis
Employee (<code>base.group_user</code>)	read only	read, create and edit own requests	no — raises an Access Error	no
Officer (<code>hr.group_hr_user</code>)	full, no delete	full, no delete	yes	no
Administrator (<code>hr.group_hr_manager</code>)	full incl. delete	full incl. delete	yes	yes

Two things worth knowing:

- **"Own requests" is enforced by a record rule**, not by a filter: `[('employee_id.user_id', '=', user.id)]`. A regular employee cannot read a colleague's request even by URL or RPC.
- **The approval family is blocked in Python**, not just hidden in the UI. Calling `action_approve()`, `action_returned()` or the refuse wizard as a plain employee — via RPC, an automated action or a shell — raises `AccessError`. Hiding the buttons alone would not have been enough.

Both models also ship a **multi-company record rule** (`company_id in company_ids`), so a user only ever sees the items and requests of the companies they are allowed into.

4. Where to find it

Employees → **Custody**, with three entries:

Menu	Opens	Visible to
Custody Requests	list / form of <code>odomate.hr.custody</code>	every internal user
Items	kanban / list / form of <code>odomate.hr.custody.item</code>	every internal user

Analysis

pivot over `odomate.hr.custody`

HR Administrator

5. Managing the item register

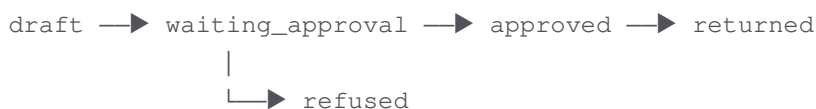
Employees → Custody → Items opens a kanban board. Each card shows the item photo (or a neutral cube placeholder when no photo is uploaded), the item name, and a status badge — green **Available** or amber **Held by name**. A small grey **Product** badge appears only when the item is linked to a product record.

On the item form:

- **Item Name** — required, e.g. `Dell Latitude 5540 (SN DL-5540-0031)`.
- **Photo** — optional.
- **Description** — free text: serial number, accessories, condition at purchase.
- **Company** — required; shown only when multi-company is enabled.
- **Related Product** — optional link to `product.product`. Picking a product fills the **Item Name** once, as a convenience. The two fields are **not** kept in sync afterwards, so renaming the product later does not rename the item.
- **Available / Current Holder / Current Custody** — computed live from the request history; you never edit them.

The **Custody History** smart button opens every request ever filed for that item.

6. The request workflow



Button	Visible when	Who
Send for Approval	status is Draft	requester
Approve	status is Waiting Approval	Officer
Refuse	status is Waiting Approval	Officer
Set to Draft	Waiting Approval or Refused, and no extension pending	requester
Extend	status is Approved	holder
Returned	status is Approved	Officer
Send Reminder	status is Approved	Officer



Print Handover Document	Approved or Returned	anyone who can see the record
-------------------------	----------------------	-------------------------------

The reference

Every request gets a reference from a single global counter with 5-digit padding: `CUST/00001`, `CUST/00002`, ... The counter never resets — not per year, not per company.

One holder at a time

An item can be in exactly one approved request. Approving a second request for the same item fails with a named message:

Dell Latitude 5540 (SN DL-5540-0031) is already in the custody of Mitchell Admin under request CUST/00001. Close that request before approving this one.

This is checked inside `action_approve()` itself, so an import, an RPC call or a server action hits the same guard as the button. It is additionally backed by a **partial unique database index** on `(item_id)` `WHERE state = 'approved'`, so two officers approving simultaneously cannot both win the race.

Returning an item

Returned stamps **Actual Return Date** with today's date and moves the request to Returned. It never overwrites the promised **Return Date** — the two dates stay side by side so you can see how late the return actually was.

Set to Draft is deliberately restricted

Set to Draft is available only for a request that has never been approved. Once a request reaches Approved, or while an extension is pending, the button disappears **and** `action_set_to_draft()` raises a `UserError` — so the restriction also holds for RPC and imports.

7. Extending a return date

While an item is Approved, the holder clicks **Extend** and enters a new date. The wizard rejects any date on or before the **Request Date**.

On confirmation:

- the proposed date is stored in **Proposed Return Date** — the original **Return Date** is left untouched;
- **Pending Extension** is set;
- the request goes back to Waiting Approval.

The form then shows both dates side by side, e.g. `Return Date 12/09/2026 requested: 03/10/2026`.

An extension has exactly two outcomes:



- **Approve** — the proposed date replaces **Return Date**, both extension fields are cleared, and the request returns to Approved.
- **Refuse** — the reason is stored in **Extension Refusal Reason**, the proposed date is discarded, and the request returns to Approved with the **original** return date. There is no separate "cancel extension" action; refusing is the only exit, and it is the only one that records a reason.

8. Refusals

Both refusal paths use the same wizard, which requires a reason, but they write to **different fields** and are labelled separately on the form:

Situation	Field written	Resulting status
Refusing an initial request	Refusal Reason	Refused
Refusing an extension	Extension Refusal Reason	back to Approved

The wizard is bound solely to `odomate.hr.custody` — it does not read a model name from the context.

9. Overdue reminders

A request is **overdue** when it is Approved and its **Return Date** is in the past. Overdue rows are highlighted in red in the request list, and the form shows a warning banner.

Automatic chasing. The scheduled action Custody: Daily Return Reminders runs once a day at 05:00. It selects every request where `state = approved AND return_date <= today + 1 day` and emails the holder using the **Custody: Return Reminder** template — the same template the manual **Send Reminder** button uses.

The cron deliberately re-sends every day for as long as the item is out. This is an intentional exception to the usual "notify once on the state transition" rule: the business requirement is to keep nagging, not to fire once. The recipient is the holder only — there is no escalation to a manager.

The email contains a direct link to the request, built from `get_base_url()` in Odoo 19's current record-address form (`/odoo/action-odomate_hr_custody.action_odomate_hr_custody/<id>`). No host is hard-coded.

If an employee has neither a linked user nor a work contact, no email can be addressed to them: the cron skips them silently, and the manual button reports the problem instead of failing quietly.

The schedule is fixed and deliberately not exposed in Settings. An administrator who needs a different hour can change it under **Settings → Technical → Scheduled Actions**.

10. The handover document

Print Handover Document produces a one-page PDF on Odoo's standard external layout, so your company heading, logo and address come from `res.company`. It is available from the request form and from a list selection (⚙ → **Print** → **Custody Handover Document**).



It contains, in order:

1. Reference and request date.
2. The holder's name, job position, department, identification reference, emergency contact and emergency phone.
3. The item name and description.
4. The reason for the request and the return date.
5. The undertaking text the employee signs.
6. Two signature lines — Employee signature and Authorised by (HR).

Employee fields that are empty print as a blank line. They never raise an error and never print the literal word `False`.

11. Analysis and employee smart buttons

Employees → Custody → Analysis (HR Administrator only) opens a pivot over `odomate.hr.custody` itself — there is no separate reporting model. Items are on rows and status on columns by default; the search panel lets you group by **Employee**, **Item**, **Status** and **Request Month**.

On any employee form, two smart buttons appear for HR users:

- **Custody** — every request ever filed by that employee.
- **Held Now** — only the requests currently in the Approved state.

12. Field reference

`odomate.hr.custody`

Field	Type	Notes
<code>name</code>	Char	Reference, read-only, from the <code>odomate.hr.custody</code> sequence
<code>employee_id</code>	Many2one <code>hr.employee</code>	Required; defaults to the current user's employee
<code>item_id</code>	Many2one <code>odomate.hr.custody.item</code>	Required
<code>company_id</code>	Many2one <code>res.company</code>	Required; defaults to the active company
<code>reason</code>	Char	Required
<code>request_date</code>	Date	Required; defaults to today
<code>return_date</code>	Date	Required; the promised date



actual_return_date	Date	Read-only; set only by Returned
state	Selection	draft, waiting_approval, approved, returned, refused; tracked
is_extension	Boolean	Stored; true while an extension awaits approval
extend_new_return_date	Date	The proposed date, held apart from return_date
refusal_reason	Text	Plain refusal
extension_refusal_reason	Text	Extension refusal — never shares a field with the above
is_overdue	Boolean	Computed, not stored; used for list decoration only
notes	Text	Free text

odomate.hr.custody.item

Field	Type	Notes
name	Char	Required
image_1920	Image	Optional
description	Text	
company_id	Many2one res.company	Required
product_id	Many2one product.product	Optional, no domain restriction
custody_ids	One2many	All requests for this item
current_custody_id	Many2one	Computed, not stored
current_holder_id	Many2one hr.employee	Computed, not stored
is_available	Boolean	Computed, not stored

hr.employee (added fields)

Field	Type	Notes
custody_count	Integer	Computed, not stored
custody_current_count	Integer	Computed, not stored

13. Limitations

Deliberately **not** included in this module:



- **No quantities or stock integration.** One register row is one physical item; the optional `product_id` link is descriptive only and moves nothing in Inventory.
- **No condition, damage or maintenance tracking**, and no repair workflow.
- **No financial value, depreciation or insurance fields.**
- **No multi-level approval.** One officer approval is final.
- **No escalation.** Reminders go to the holder only, never to a manager.
- **No barcode scanning** and **no on-screen signature capture** — the handover document is printed and signed on paper.
- **No per-item history model.** The request records are the history.
- **The reminder schedule is fixed** at daily 05:00 with no settings screen.
- **`is_overdue` is not stored**, so you cannot group or filter on it directly. Use the **Overdue** search filter, which evaluates the dates in the domain.
- **Regular employees cannot delete their own requests.** Ask an HR Administrator to remove a mistaken record.

Generated by OdoMate · <https://odomate.pro> · support@odomate.pro

